

Java Strings

A Concept Introduction

Learn how Java Strings really work — not just the syntax, but the reasoning behind it

Best read alongside a code editor, so you can run every example yourself

What You'll Learn

#	Topic
1	String Operations and Performance
2	Built-in String Methods — <code>length()</code> , <code>charAt()</code> , <code>substring()</code> , <code>indexOf()</code> , and more
3	String Manipulation Methods — <code>trim()</code> , <code>replace()</code> , <code>split()</code> , and more
4	ASCII Character Codes — understanding and converting character values
5	Mutable String Handling — <code>StringBuilder</code> and <code>StringBuffer</code>
6	String Performance Optimization — writing efficient String code

1. String Operations and Performance

A String in Java is simply a sequence of characters used to represent text — anything from a single word to an entire paragraph. You've likely already used them, perhaps without thinking too much about it:

```
String greeting = "Hello, World!";
```

What's less obvious is what happens to a String once it's created. Java treats Strings as immutable — meaning once a String is created, its content can never be changed. Every time you "modify" a String, Java is actually creating a brand-new String behind the scenes and leaving the original untouched.

Why This Matters

WHY THIS MATTERS

- **Safety:** Strings are used for things like file paths, class names, and network addresses. If they could be changed after being checked, that would open the door to security bugs.
- **Speed:** Since content never changes, Java can safely let many variables share a single copy of identical text, instead of storing duplicates (this is called the "String Pool").
- **Thread-safety:** An object that can never change can be freely shared across different parts of a program — even different threads — with zero risk of one part accidentally corrupting it for another.

PAUSE & THINK: *If Strings could be changed after creation, why might sharing the same String value across many parts of a large program become risky?*

Try This Code

```
public class StringBasicsDemo {
    public static void main(String[] args) {
        // Two identical literal Strings are automatically stored as ONE
        // shared object in memory, called the String Pool.
        String a = "Java";
        String b = "Java";
        System.out.println(a == b);           // true -- same shared object

        // Using 'new' explicitly creates a separate object, even with
        // identical text -- it does NOT use the shared pool.
        String c = new String("Java");
        System.out.println(a == c);           // false -- different object

        // .equals() compares the ACTUAL TEXT, regardless of memory location.
        System.out.println(a.equals(c));      // true -- same content

        // "Changing" a String really means creating a brand-new one.
        String greeting = "Hello";
        greeting = greeting + " World";       // "Hello" itself is untouched
        System.out.println(greeting);         // Hello World
    }
}
```

Output

```
true
false
true
Hello World
```

WATCH OUT FOR

- Using `==` to compare String content instead of `.equals()` — this can accidentally seem to work while testing,

then fail later with real data (like user input or database values).

- Writing new `String("text")` without a real reason — it just creates an extra, unnecessary object.

Before You Move On

- A `String` is a sequence of characters representing text.
- Strings are immutable — every "edit" actually creates a new object.
- Always use `.equals()` to compare `String` content, not `==`.

2. Built-in String Methods

Java gives you a rich set of built-in methods for reading and inspecting String content. Think of a String as a row of numbered boxes, each holding one character, starting from position 0:

Index:	0	1	2	3	4
Character:	H	e	l	l	o

length()

Returns the total number of characters in the String — including spaces.

```
String word = "Hello";  
System.out.println(word.length()); // 5
```

charAt()

Returns the single character at a given position (index). Remember: counting starts at 0, so the last valid index is `length() - 1`.

```
System.out.println(word.charAt(0)); // H  
System.out.println(word.charAt(word.length() - 1)); // o
```

substring()

Extracts part of a String. `substring(start)` returns everything from `start` to the end. `substring(start, end)` returns characters from `start` up to — but not including — `end`.

```
System.out.println(word.substring(2)); // "llo"  
System.out.println(word.substring(1, 4)); // "ell"
```

indexOf()

Searches for the first position where a character or substring appears. If it isn't found, it returns -1 instead of raising an error.

```
System.out.println(word.indexOf('l')); // 2 (first match)  
System.out.println(word.indexOf("xyz")); // -1 (not found)
```

Other Commonly Used Methods

Beyond the four above, a handful of other methods come up constantly in everyday code:

- `contains(text)` — checks whether a substring exists anywhere inside the String.
- `toUpperCase()` / `toLowerCase()` — return a new String with the case changed.
- `equalsIgnoreCase()` — compares content while ignoring uppercase/lowercase differences.
- `isEmpty()` / `isBlank()` — check whether a String has no characters (`isBlank()` also treats whitespace-only text as empty).
- `startsWith()` / `endsWith()` — check the beginning or end of a String.

PAUSE & THINK: Given that `substring(1, 4)` stops BEFORE index 4, what do you think `substring(0, word.length())` would return?

Try This Code

```
public class StringMethodsDemo {  
    public static void main(String[] args) {
```

```
String city = "Chennai";

System.out.println(city.length());           // 7
System.out.println(city.charAt(0));          // C
System.out.println(city.substring(3));       // nnai
System.out.println(city.substring(1, 5));    // henn
System.out.println(city.indexOf('n'));       // 2
System.out.println(city.indexOf("xyz"));     // -1 (not found)

System.out.println(city.contains("hen"));    // true
System.out.println(city.toUpperCase());      // CHENNAI
System.out.println(city.equalsIgnoreCase("CHENNAI")); // true
System.out.println("").isEmpty();           // true
System.out.println(city.startsWith("Che")); // true
}
```

Output

```
7
C
nnai
henn
2
-1
true
CHENNAI
true
true
true
```

WATCH OUT FOR

- Forgetting that `substring(start, end)` excludes the character at 'end' — a very common off-by-one mistake.
- Using an `indexOf()` result without checking for -1 first, which can cause errors later in your code.

Before You Move On

- Indexing always starts at 0.
- `substring(start, end)` excludes the character at 'end'.
- None of these methods change the original String — they all return a new value.

3. String Manipulation Methods

While the methods in Topic 2 let you inspect a String, this next set lets you transform it — cleaning it up, replacing parts of it, or breaking it into pieces.

trim()

Removes any whitespace from the very start and end of a String. It does NOT touch spaces in the middle.

```
String messy = " Hello Java ";  
System.out.println "[" + messy.trim() + "]" ; // [Hello Java]
```

replace()

Replaces every occurrence of a character or piece of text with another. It looks for an exact, literal match.

```
String sentence = "I like cats";  
System.out.println(sentence.replace("cats", "dogs")); // I like dogs
```

split()

Breaks a String apart into an array of pieces, based on a separator you provide. This is one of the most useful methods for working with structured text like CSV data.

```
String csv = "Ravi,25,Chennai";  
String[] parts = csv.split(",");  
// parts = ["Ravi", "25", "Chennai"]
```

One thing to know: `split()` treats its argument as a pattern (called regex), not just plain text. Symbols like `.` or `|` have special meaning, so splitting on a literal period requires escaping it with `split("\\. ")`.

Additional Useful Manipulation Methods

- `concat()` — joins two Strings together, similar to using `+`.
- `String.join(separator, ...)` — joins multiple Strings using a chosen separator.
- `toCharArray()` — converts a String into an array of individual characters, useful when you need to loop through them one by one.

PAUSE & THINK: `"a,b,.".split(",")` returns only 2 items, not 4 — Java quietly drops empty pieces at the end. Why do you think it was designed to work that way?

Try This Code

```
public class StringManipulationDemo {  
    public static void main(String[] args) {  
        System.out.println(" Hello Java ".trim()); // "Hello Java"  
        System.out.println("I like cats".replace("cats", "dogs")); // "I like dogs"  
  
        String[] parts = "Ravi,25,Chennai".split(",");  
        for (String part : parts) {  
            System.out.println(part);  
        }  
  
        // concat() joins two Strings together  
        System.out.println("Hello".concat(" World"));  
  
        // String.join() stitches multiple pieces back together  
        System.out.println(String.join("-", "2026", "07", "30"));
```

```
// toCharArray() breaks a String into individual characters
char[] chars = "Java".toCharArray();
for (char c : chars) {
    System.out.print(c + " ");
}
}
```

Output

```
Hello Java
I like dogs
Ravi
25
Chennai
Hello World
2026-07-30
J a v a
```

WATCH OUT FOR

- Splitting on a symbol like '.' without escaping it — `split(".")` does not behave the way most people expect at first.
- Using raw `split()` on a real CSV file — a comma inside a quoted value (like "Smith, John") will break the result. Larger projects usually use a dedicated parsing library instead.

Before You Move On

- `trim()` only cleans the edges of a String, never the middle.
- `replace()` looks for exact text; `split()` uses pattern matching, so handle it carefully.
- None of these methods modify the original String — they all return a new one.

4. ASCII Character Codes

Every character your program works with — letters, digits, symbols — is stored internally as a number. ASCII (American Standard Code for Information Interchange) is the classic standard that assigns a number to each of the 128 basic English characters.

Understanding ASCII Values

Uppercase letters A–Z run from 65 to 90. Lowercase letters a–z run from 97 to 122. Digits '0'–'9' run from 48 to 57. Notice that uppercase and lowercase letters are always exactly 32 apart:

```
'A' -> 65      'a' -> 97
'B' -> 66      'b' -> 98
'Z' -> 90      'z' -> 122
```

Character-to-ASCII Conversion

In Java, a char is actually stored as a number under the hood. Casting a char to an int retrieves that numeric value:

```
char letter = 'A';
int code = (int) letter;
System.out.println(code); // 65
```

ASCII-to-Character Conversion

You can go the other way too — casting an int back to char converts a number back into its character:

```
int code = 97;
char letter = (char) code;
System.out.println(letter); // a
```

WHY THIS MATTERS

- ASCII is only the first 128 characters of a much larger system called Unicode, which Java actually uses for all of its Strings — this is what lets Java represent almost any language, symbol, or emoji, not just English text.
- Because uppercase and lowercase letters are always 32 apart, you can convert between them with simple math — a handy trick, though real projects usually prefer the built-in Character class for this (shown below).

PAUSE & THINK: *Since Java can represent way more than 128 characters, what do you think happens when a single character (like certain emoji) needs more "space" than a basic character does?*

Try This Code

```
public class AsciiDemo {
    public static void main(String[] args) {
        // Character to ASCII
        char letter = 'A';
        System.out.println((int) letter);           // 65

        // ASCII to character
        char back = (char) 97;
        System.out.println(back);                   // a

        // Uppercase -> lowercase using the 32-gap trick
        char upper = 'M';
        char lower = (char) (upper + 32);
        System.out.println(lower);                  // m

        // Printing the whole alphabet using ASCII values
        for (int i = 65; i <= 90; i++) {
```



```
        System.out.print((char) i);  
    }  
    System.out.println();  
  
    // The safer, built-in alternative for real projects  
    System.out.println(Character.isDigit('7')); // true  
    System.out.println(Character.isLetter('Z')); // true  
    System.out.println(Character.toUpperCase('m')); // M  
}  
}
```

Output

```
65  
a  
m  
ABCDEFGHIJKLMNOPQRSTUVWXYZ  
true  
true  
M
```

WATCH OUT FOR

- Using manual math like +32 on text that might not be plain English letters — it can produce incorrect results for other languages or symbols.
- Assuming every character always fits neatly into a simple number — some characters (like emoji) need more space than a basic character provides.

Before You Move On

- Every character maps to a number; ASCII covers the first 128 of them.
- Cast char to int to get the numeric code; cast int to char to reverse it.
- Prefer Character.isDigit() / isLetter() / toUpperCase() over manual math in real code.

5. Mutable String Handling

You now know that String is immutable — every change creates a new object. That's usually fine, but it becomes a problem when you need to build or edit text many times in a row, like inside a loop. That's exactly what StringBuilder and StringBuffer are for.

StringBuilder

StringBuilder is a mutable alternative to String — you can add, insert, delete, or replace its content directly, without creating a new object every time.

```

StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");    // Hello World -- modifies the SAME object
sb.insert(5, ",");      // Hello, World
sb.replace(0, 5, "Hi"); // Hi, World
sb.delete(2, 3);        // Hi World
System.out.println(sb); // Hi World
  
```

StringBuffer

StringBuffer has the exact same set of methods as StringBuilder. The only difference is that StringBuffer is thread-safe — meaning it's built to be used safely when multiple parts of a program write to it at the same time. That extra safety comes with a small performance cost.

```

StringBuffer safe = new StringBuffer("Thread-safe");
safe.append(" text");
System.out.println(safe); // Thread-safe text
  
```

Differences Between String, StringBuilder, and StringBuffer

	String	StringBuilder	StringBuffer
Can it change?	No (immutable)	Yes (mutable)	Yes (mutable)
Safe with multiple threads?	N/A	No	Yes
Speed	Slow to modify repeatedly	Fast	Slightly slower (thread-safety adds cost)
Best used for	Fixed or rarely-changed text	Building/editing text (default choice)	Text shared and edited across threads

WHY THIS MATTERS

- StringBuilder keeps one growable block of memory and simply adds to it — this avoids the repeated copying that happens with regular String concatenation.
- Because it doesn't need to worry about multiple threads, StringBuilder can skip the extra safety checks that StringBuffer performs — which is exactly why it's faster.

PAUSE & THINK: *If your program is single-threaded (most student projects are), is there any real benefit to using StringBuffer over StringBuilder?*

WATCH OUT FOR

- Using StringBuffer "just to be safe" in ordinary single-threaded code — it adds cost for no real benefit.
- Creating a brand-new StringBuilder inside a loop instead of reusing one — this defeats the whole purpose of using it.

Before You Move On

- String = immutable. StringBuilder and StringBuffer = mutable.
- Default to StringBuilder — it's the right choice almost all the time.
- Use StringBuffer only when multiple threads truly share and edit the same buffer.

6. String Performance Optimization

Why String Concatenation Can Be Inefficient

Because Strings are immutable, every time you use += inside a loop, Java doesn't edit the existing text — it builds an entirely new String, copying everything that came before it, plus the new piece. Do that thousands of times in a loop, and you're copying the same growing text over and over again.

```
String result = "";
result += "a";    // creates NEW object: "a"                (copies 0 chars)
result += "b";    // creates NEW object: "ab"   (copies "a") (copies 1 char)
result += "c";    // creates NEW object: "abc"   (copies "ab") (copies 2 chars)
// ...and so on -- the amount of copying grows with every step
```

When to Use StringBuilder and StringBuffer

- A single-line concatenation like "Hi " + name + "!" is perfectly fine — Java already handles this efficiently on its own.
- Building text across many steps in a loop should almost always use StringBuilder instead.
- Reach for StringBuffer only in the rare case where multiple threads genuinely share and write to the same buffer at once.

WHY THIS MATTERS

- StringBuilder solves this problem by reusing the SAME block of memory as it grows, instead of creating a new String at every step.
- The performance gap between + and StringBuilder isn't small — it gets dramatically worse as the loop gets longer, because more and more text has to be re-copied each time.

PAUSE & THINK: *If Java already optimizes a single line like "Hi " + name, why doesn't that same optimization help when the concatenation happens inside a loop?*

Performance Comparison Example

```
public class PerformanceDemo {
    public static void main(String[] args) {
        int n = 50_000; // number of concatenations to perform

        // SLOW: creates a brand-new String on every single loop iteration
        String result = "";
        long t1 = System.nanoTime();
        for (int i = 0; i < n; i++) { result += "x"; }
        long time1 = (System.nanoTime() - t1) / 1_000_000;

        // FAST: reuses one growable buffer instead of copying every time
        StringBuilder sb = new StringBuilder();
        long t2 = System.nanoTime();
        for (int i = 0; i < n; i++) { sb.append("x"); }
        long time2 = (System.nanoTime() - t2) / 1_000_000;

        System.out.println("String +=      : " + time1 + " ms");
        System.out.println("StringBuilder : " + time2 + " ms");
    }
}
```

Output (try it yourself — your numbers will vary by machine)

```
String +=      : 850 ms
StringBuilder  : 3 ms
```

WATCH OUT FOR

- Using StringBuilder for a tiny, one-off concatenation "just in case" — unnecessary, and plain + is easier to read there.
- Leaving += inside a loop that processes a lot of data (like reading a large file line by line) — this is exactly the pattern that causes real slowdowns.

Golden Rule to Remember

- One-line concatenation ("Hi " + name) → plain String is fine.
- Building text inside a loop → always use StringBuilder.

Wrap-Up

Cheat Sheet

Topic	Remember This
Strings & Performance	Strings never change — every edit creates a new object.
Built-in Methods	length(), charAt(), substring(), indexOf() — read-only, all start counting at 0.
Manipulation Methods	replace() = exact text; split() = pattern-based, handle with care.
ASCII	The first 128 characters of the larger Unicode set Java uses; char <-> int casting converts between them.
StringBuilder / StringBuffer	Mutable alternatives to String; default to StringBuilder unless multiple threads share the buffer.
Performance	Never concatenate with + inside a loop — use StringBuilder.

Test Yourself

Try answering these from memory before flipping back through the guide. If you get stuck, that tells you exactly what to revisit.

1. Why does == sometimes say two identical-looking Strings are "different"?
2. Why does substring(1, 4) NOT include the character at index 4?
3. Why is split() a little riskier to use than replace()?
4. What's the numeric gap between an uppercase and lowercase version of the same letter in ASCII?
5. When is StringBuffer actually the right tool — and why not by default?
6. Why does + inside a loop stay slow even though Java optimizes simple concatenation?

Practice on Your Own

7. Write a program that checks if a word is a palindrome, using charAt() in a loop.
8. Build a simple Caesar cipher that shifts each letter of a word forward by 1, using ASCII math.
9. Write a sentence reverser using split() and StringBuilder.
10. Parse a CSV-formatted line into separate fields using split(), and print each field on its own line.